SQL Query Explanation Report

WorkForcePro — Workforce Management & OJT Platform
Database Validation

Project	WorkForcePro Workforce Management & OJT Platform
Document	SQL Query Explanation Report
Dialect	PostgreSQL
Contents	6 reporting queries + 3 simple data checks
Companion File	WorkForcePro_SQL_Validation.sql
Execution Status	Queries written and ready; not run against a live database
Date	27 June 2026

The tables being queried:

```
employees(id, employee_id, name, email, department)
trainings(id, training_name) employee_trainings(id,
employee_id, training_id, status [, due_date])
certifications(id, employee_id, certification_name,
expiry_date)
```

A training counts as done when its status is 'Completed'.

1. The Queries

---

1. All employees and their departments

What it answers: Lists every employee together with the department they belong to. (Requirement: EMP-003)

```
SELECT employee_id,
name,
department FROM
employees
ORDER  BY department, name;
```

How it works: The department is stored directly on the employees table, so we just select the columns. Sorting by department and then by name makes the list easy to scan.

What we'll see: One row per employee showing Employee ID, Name and Department.

2. Employees who have completed ALL their trainings

What it answers: Finds employees whose trainings are all marked 'Completed' (none still assigned or pending).
(Requirement: OJT-002)

```
SELECT e.employee_id,
       e.name
FROM   employees
e
JOIN   employee_trainings et ON et.employee_id = e.id
GROUP  BY e.employee_id, e.name
HAVING COUNT(*) = SUM(CASE WHEN et.status = 'Completed' THEN 1 ELSE 0 END);
```

How it works: For each employee we look at two numbers: how many trainings they have in total (COUNT(*)), and how many of those are 'Completed' (the SUM line adds 1 for each completed row). If the two numbers are equal, every training is complete, so the employee is included. The JOIN means employees with no trainings are naturally left out.

What we'll see: One row per employee who has finished all of their assigned trainings.

3. Number of trainings per employee

What it answers: Shows how many trainings each employee has been assigned. (Requirement: OJT-001)

```
SELECT e.employee_id,
       e.name,        COUNT(et.id)
AS total_trainings
```

```
FROM   employees e
LEFT   JOIN employee_trainings et ON et.employee_id = e.id
GROUP  BY e.employee_id, e.name
ORDER  BY total_trainings DESC;
```

How it works: A LEFT JOIN keeps every employee, even those with no trainings (they show a count of 0). COUNT(et.id) counts the training rows for each employee, and we sort from the most trainings to the least.

What we'll see: One row per employee with a total_trainings count, highest first.

4. Certifications expiring within 30 days

What it answers: Lists certifications that will expire within the next 30 days. (Requirement: CERT-003)

```
SELECT e.name,
       c.certification_name,
       c.expiry_date
FROM   certifications
c
JOIN   employees e ON e.id = c.employee_id
WHERE  c.expiry_date BETWEEN CURRENT_DATE AND CURRENT_DATE + 30
ORDER  BY c.expiry_date;
```

How it works: We keep only certificates whose expiry date falls between today and 30 days from today. BETWEEN includes both ends, so a certificate expiring in exactly 30 days IS shown - this is the correct, inclusive reading of the rule. (The application currently misses the exact 30-day case; see DEF-004.)

What we'll see: Employee name, certification name and expiry date for each soon-to-expire certificate, earliest first.

5. Employees with overdue trainings

What it answers: Lists trainings that are past their due date and not yet completed. (Requirement: OJT-001)

```
SELECT e.name,
       t.training_name,
et.due_date FROM
employee_trainings et
JOIN   employees e ON e.id = et.employee_id
JOIN   trainings  t ON t.id = et.training_id
WHERE  et.status <> 'Completed'
AND    et.due_date < CURRENT_DATE;
```

How it works: We keep training rows that are not yet 'Completed' and whose due date is earlier than today that is what 'overdue' means. Joining to employees and trainings adds readable names. Note: the schema provided for employee_trainings does not list a due_date column; this query assumes one exists (the app captures a Due Date at assignment). If the app instead stores an 'Overdue' status, use WHERE et.status = 'Overdue'.

What we'll see: Employee name, training name and due date for each overdue training.

6. Top 5 employees by completed trainings

What it answers: Shows the five employees who have completed the most trainings. (Requirement: OJT-002)

```
SELECT e.employee_id,
       e.name,        COUNT(*) AS
completed_trainings
FROM   employees e
JOIN   employee_trainings et ON et.employee_id = e.id
WHERE  et.status = 'Completed'
GROUP  BY e.employee_id, e.name
ORDER  BY completed_trainings DESC
LIMIT  5;
```

How it works: First we keep only the 'Completed' training rows (the WHERE line). Then we count them per employee, sort from highest to lowest, and keep just the top five with LIMIT 5.

What we'll see: Up to five rows, each showing the employee and their completed_trainings count, highest first.

2. Simple Data Checks (find the reported issues)

These short queries check the same business rules from the data side and reveal the logged defects.

V1. Find duplicate emails (ignoring upper/lower case) - validates EMP-002 / DEF-001

```
SELECT LOWER(email) AS email,
       COUNT(*)      AS times_used
FROM   employees
GROUP  BY LOWER(email)
HAVING COUNT(*) > 1;
```

How it works: We group employees by their lower-cased email and keep any group used more than once. Any row here means the same email exists more than once in different letter case - the data behind DEF-001.

V2. Find duplicate Employee IDs - validates EMP-001 / DEF-006

```
SELECT employee_id,
       COUNT(*) AS times_used
FROM   employees
GROUP  BY employee_id
HAVING COUNT(*) > 1;
```

How it works: We group by Employee ID and keep any ID used by more than one record. Any row here is a duplicate Employee ID - the data behind DEF-006.

V3. Certifications expiring in exactly 30 days - validates CERT-003 / DEF-004

```
SELECT e.name,
       c.certification_name,
       c.expiry_date
FROM   certifications
c
JOIN   employees e ON e.id = c.employee_id
WHERE  c.expiry_date = CURRENT_DATE + 30;
```

How it works: Lists certificates expiring exactly 30 days from today. These should appear in the app's 'expiring within 30 days' report but are currently missed by its exclusive comparison (DEF-004).

3. Notes for the Reviewer

#	Topic	Note / Recommendation	Priority
N1	due_date column	Query 5 (overdue trainings) needs a due_date on employee_trainings. The supplied schema does not list one; the query assumes it exists, matching the app's Due Date field. Recommend adding due_date DATE.	High
N2	approval evidence	There is no approved_by / approved_at column, so the database cannot prove a 'Completed' training was actually approved (OJT-003). Recommend adding these for an audit trail.	Medium
N3	email uniqueness	Add a UNIQUE index on LOWER(email) so duplicate emails (DEF-001) cannot be saved.	High
N4	employee_id uniqueness	Add a UNIQUE constraint on employee_id so duplicate IDs (DEF-006) cannot be saved.	Medium